

Towards Proactive Detection of User-Side Implicit Conflicts in Human–LLM Dialogue

Jinqiang Wang^{a,1}, Tao Zhu^b and Huansheng Ning^{a,*}

^a*School of Computer & Communication Engineering, University of Science and Technology Beijing, Beijing, 100083, China*

^b*School of Computer Science, University of South China, Hengyang, 421001, China*

ARTICLE INFO

Keywords:

Agent

LLM

Data Synthesis

User-side Conflict

Implicit Conflict

Memory

Proactive Interaction

ABSTRACT

In human–LLM dialogue, follow-up user utterances may implicitly conflict with earlier intents, leading the LLM to misinterpret user needs and generate inappropriate responses. A reliable dialogue system should proactively detect user-side conflicts before generating a response and seek clarification when necessary. However, prior work has largely focused on LLM-side conflicts, leaving user-side conflicts underexplored. To fill this gap, we construct UC-Bench, a human-annotated benchmark for evaluating user-side conflict detection. Preliminary experiments show that existing LLMs struggle with this task, especially when conflicts arise from implicit incompatibilities grounded in dialogue history. To improve lightweight LLMs with limited training data, we investigate data synthesis for user-side conflict detection. Existing synthesis methods do not explicitly model the implicit incompatibilities between historical and current user utterances, making it difficult to capture the evolution of conflicts and to generate reliably labeled implicit conflict samples. We propose SynUC, a constraint-guided synthesis method that represents user-side conflicts in a constraint space and uses the SPEAKING framework to guide traceable constraint transformations. Applying SynUC to WildChat, we construct UC-Data, a user-side conflict training set containing 2,487 samples. On UC-Bench, Qwen3.5-4B trained on UC-Data outperforms larger general-purpose LLMs such as Claude Opus 4.8, as well as the same backbone trained on data synthesized by existing methods.

1. Introduction

Large language models (LLMs) are increasingly used for code generation (Seo et al., 2026) and document writing (Mysore et al., 2025). However, most existing systems still follow a passive interaction paradigm (Wu et al., 2026; Wang et al., 2025a; Zhang et al., 2026; Li et al., 2025c) and respond to explicit requests without assessing whether newly introduced requirements are compatible with those established earlier in the dialogue. In human–LLM interactions, users often specify their needs incrementally by supplementing, revising, or shifting their intents across turns (Li et al., 2025b). Without proactively identifying incompatibilities introduced during this process, an LLM may simply follow the latest utterance and generate responses that deviate from the user’s underlying intent.

Recent studies have begun to examine instruction conflicts in human–LLM dialogue. ConInstruct (He et al., 2026) focuses on conflicts among constraints within a single user utterance, while MultiTurnInstruct (Han et al., 2025) evaluates models’ abilities to retrieve information, track dialogue states, and resolve conflicts in multi-turn entangled instructions. However, these studies mainly assess instruction-following behavior or response quality, leaving implicit conflict relations in user-side contexts underexplored. In this work, implicit conflict refers to a situation in which a user’s new requirement is incompatible with earlier requirements, but the user does not explicitly indicate a revision.

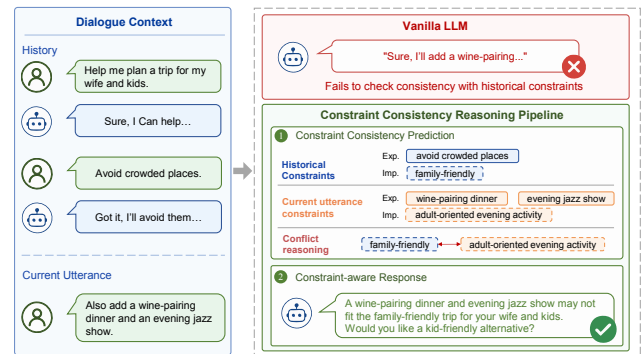


Figure 1: Comparison between a vanilla LLM and a constraint reasoning pipeline when handling an utterance containing an implicit conflict.

As shown in Figure 1, the user first asks the LLM to “plan a trip for my wife and kids” and further specifies that it should “avoid crowded places.” This establishes an explicit constraint to avoid crowded locations and an implicit constraint that the itinerary should be family-friendly. However, the follow-up utterance requests a “wine-pairing dinner” and a “late-night jazz show.” Although the utterance does not explicitly revise the target participants, the requested activities may conflict with the earlier family-friendly goal. Detecting this conflict allows the LLM to request clarification rather than directly incorporating activities that overlook the family-friendly constraint.

To evaluate the ability of LLMs to detect user-side implicit conflicts, we first construct **UC-Bench**, a human-annotated benchmark. UC-Bench categorizes the relation

*Corresponding author

✉ jqwang@xs.ustb.edu.cn (J. Wang); tzhu@usc.edu.cn (T. Zhu);

ninghuansheng@ustb.edu.cn (H. Ning)

ORCID(s): 0000-0002-3711-0743 (J. Wang); 0000-0002-5879-5980 (T. Zhu); 0000-0001-6413-193X (H. Ning)

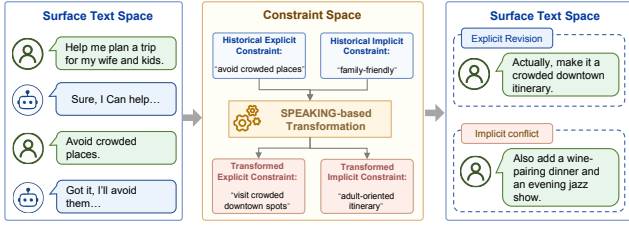


Figure 2: Illustration of the constraint-guided synthesis idea of SynUC.

between the current user utterance and the dialogue history as normal dialogue, explicit revision, or implicit conflict. Normal dialogue indicates that the current utterance is compatible with previous requirements, whereas explicit revision involves an explicit modification, replacement, or cancellation of those requirements.

Based on UC-Bench, we conduct a preliminary study. The results show that LLMs often fail to recognize the underlying incompatibility when directly responding to user utterances containing implicit conflicts. In contrast, their detection performance improves when they are explicitly prompted to examine potential conflicts between the current utterance and the dialogue history. These findings motivate us to formulate user-side conflict detection as a standalone task performed by an external monitoring model, which assesses this relation before response generation.

To address the scarcity of training data for user-side conflict detection, we investigate the automatic synthesis of high-quality training samples. Existing data synthesis methods (Han et al., 2025; He et al., 2026; Hellwig et al., 2025) mainly focus on general instruction generation, task response generation, or explicit conflict construction. They lack structured modeling of implicit incompatibilities between the dialogue history and the current user utterance, making it difficult to capture conflict evolution and generate controllable implicit conflict samples. To address this limitation, we propose **SynUC**, a constraint-guided data synthesis method for user-side conflict detection. As shown in Figure 2, SynUC first maps user instructions from **surface text space** to **constraint space**, where it performs constraint transformation to derive conflict constraints. It then maps these constraints back to surface text space to form conflict-bearing follow-up user utterances. The constraint transformation module is built on Hymes’s SPEAKING framework (Hymes, 2013) and modeled as a cross-sample self-evolving memory, allowing high-quality constraint conflict construction experience to be reused. Based on WildChat, we use SynUC to construct **UC-Data**, a training set with 2,487 samples for supervised fine-tuning and reinforcement learning. To further improve reinforcement learning on this task, we introduce a constraint-attribution reward that encourages the model to identify the conflicting constraints.

Experimental results show that Qwen3.5-4B, a lightweight model trained on SynUC-synthesized data, outperforms a range of strong baselines on UC-Bench, including the larger

closed-source model Claude Opus 4.8. Compared with other data synthesis methods, SynUC yields training data that more effectively improves implicit conflict recognition. Further ablation studies show that both constraint-space modeling and the SPEAKING-based memory contribute to the final performance, supporting the effectiveness of our method for user-side conflict detection.

The main contributions of this paper are as follows.

1. We present the first study of user-side implicit conflicts in multi-turn human–LLM dialogue. We construct UC-Bench, a benchmark for user-side conflict detection, and show that existing LLMs still face substantial challenges on this task.
2. We propose SynUC, a constraint-guided data synthesis method. On UC-Bench, lightweight LLMs trained on SynUC-synthesized data outperform larger general-purpose LLMs and the same backbone trained on data synthesized by existing methods.
3. We release UC-Data, UC-Bench, and the SynUC synthesis code to support future research.¹

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the task definition, key terminology, and preliminary study. Section 4 introduces the SynUC data synthesis method and the constraint-attribution reward. Section 5 describes the experimental setup. Section 6 reports the main results and analysis. Section 7 discusses key design choices and parameter effects. Section 8 concludes the paper.

2. Related Work

2.1. Dialogue Conflict Detection

Prior work on dialogue conflict detection has largely examined whether model responses are consistent with persona information, dialogue history, or external knowledge. In open-domain dialogue, this problem is often formulated as natural language inference. Dialogue NLI (Welleck et al., 2019) frames persona consistency as a natural language inference task and classifies sentence pairs derived from persona descriptions and dialogue utterances as entailment, neutral, or contradiction. DECODE (Nie et al., 2021) detects whether the final utterance contradicts the preceding dialogue context. CDConv (Zheng et al., 2022) introduces a Chinese multi-turn dialogue benchmark covering three contradiction types: intra-sentence contradiction, role confusion, and history contradiction. In task-oriented dialogue, CI-ToD (Qin et al., 2021) evaluates whether model responses are inconsistent with the user query, dialogue history, or knowledge base. Wang et al. (Wang et al., 2025b) further explore zero-shot LLM prompting and multi-agent collaboration for this task.

More recent studies investigate how LLMs handle contradictions in generated content and external knowledge.

¹The code and data are available at <https://github.com/jqwangai/SynUC>

Mündler et al. (Mündler et al., 2024) analyze self-contradiction in instruction-tuned language models under open-ended generation and question-answering settings, and propose prompt-based methods for detection and mitigation. CIDER (Zhang et al., 2024) provides inconsistent dialogue responses, natural-language explanations, and clarification-based recovery utterances to support the detection and resolution of conversational inconsistencies. Wen et al. (Wen et al., 2024) propose contradictory dialogue processing, in which models detect and explain self-contradictions and then modify the contradictory content. WikiContradict (Hou et al., 2024) evaluates whether LLMs can generate answers that accurately reflect conflicts among retrieved Wikipedia passages. From a data-generation perspective, ConsistentChat (Chen et al., 2025) uses human conversational intents and structured information flows to guide the synthesis of consistent multi-turn instruction data. In instruction following, ConInstruct (He et al., 2026) evaluates LLMs' ability to detect and resolve explicit conflicts among constraints within a single user instruction.

Overall, existing studies primarily address inconsistencies in model responses and explicit conflicts within single-turn instructions. However, they do not specifically model implicit user-side conflicts in multi-turn interactions, where a follow-up utterance conflicts with still-valid requirements established earlier in the dialogue without an explicit revision signal.

2.2. Data Synthesis

Recent work has widely used LLMs to synthesize instruction data for improving instruction following. Self-Instruct (Wang et al., 2023) bootstraps instructions, inputs, and outputs from a model itself, and uses the filtered data for instruction tuning. Baize (Xu et al., 2023) adopts a self-chat strategy, where ChatGPT plays both the user and the assistant to generate multi-turn dialogue data. UltraChat (Ding et al., 2023) further expands the scale and topic coverage of synthetic dialogue data through a systematic framework for generating large-scale, multi-topic, multi-turn instructional conversations. MAGPIE (Xu et al., 2025) shows that an aligned LLM can autoregressively generate user instructions from only a dialogue template prefix and then produce corresponding responses, enabling large-scale extraction of instruction-response data from aligned models.

Beyond scaling data, recent studies have increasingly emphasized quality control and generation logic. Evol-Instruct (Xu et al., 2024) evolves simple instructions into more complex and diverse ones through iterative rewriting, improving models' ability to follow complex instructions. REFED (Mehri et al., 2026) extracts transferable reference-level feedback from high-quality examples to guide the synthesis of new instructions and responses. DESIGNER (Liu et al., 2026) targets complex reasoning data synthesis by abstracting reusable design logic from existing high-quality questions and transferring it to new disciplinary materials or documents. Other recent methods, including GLAN (Li et al., 2025a), DS2-Instruct (Xu et al., 2026), and

CrowdSelect (Li et al., 2026), respectively improve synthetic data generation from taxonomy-driven, domain-specific, and data-selection perspectives. These studies indicate that the effectiveness of synthetic data depends not only on scale, but also on transferable feedback signals.

Beyond general instruction data synthesis, another line of work focuses on conflict scenarios in multi-turn interactions. MultiTurnInstruct (Han et al., 2025) constructs tasks involving multi-turn dependencies, information integration, and instruction conflicts to evaluate instruction following in complex multi-turn dialogue. Its contradiction resolution tasks further cover conflict-handling scenarios such as privacy protection, personalization, and prioritization. ConInstruct (He et al., 2026) focuses on explicit conflicts within a single user instruction, constructing conflict examples across six constraint dimensions: content, keyword, phrase, length, format, and style. It evaluates models' ability to detect and handle incompatible instruction requirements.

Overall, existing data synthesis methods have advanced general dialogue generation, feedback-guided synthesis, and complex reasoning data construction. However, they provide limited support for extracting relevant information from dialogue history and constructing controllable examples of implicit conflicts between historical requirements and follow-up user utterances.

3. Motivation

3.1. Definitions

We formulate **user-side conflict detection as a three-way classification task**, where each current user utterance is classified based on its consistency with previously established user requirements: normal dialogue, explicit revision, and implicit conflict.

Normal dialogue refers to cases where the current user utterance is consistent with the requirements that remain valid in the dialogue history. This category covers utterances that continue the original task, add compatible information under the original goal, or introduce a new task that does not affect existing requirements.

Explicit revision refers to cases where the user explicitly updates, replaces, or negates a previously stated requirement within the same task. The revision may be expressed through an overt signal, such as "change it to," "not ... but ...," or "ignore the previous requirement," or by directly specifying an incompatible replacement value.

Implicit conflict refers to cases where the user introduces a new requirement within the same task that is inconsistent with requirements that remain valid in the dialogue history, without explicitly updating, replacing, or negating them. Such conflicts must instead be inferred from the task goal, dialogue context, or commonsense expectations.

3.2. Preliminary Study

To preliminarily assess the ability of LLMs to recognize user-side implicit conflicts, we manually construct WildChat-UC, a small-scale evaluation subset of UC-Bench

Table 1

Resp. Acc. denotes response-based accuracy. P, R, and F1 denote macro-precision, macro-recall, and macro-F1. R(IC) denotes recall on implicit conflict samples. All values are percentages.

Model	Resp.	WildChat-UC			
	Acc.	P	R	F1	R(IC)
Claude Opus 4.8	28.33	70.91	70.00	61.71	40.00
Claude Opus 4.7	21.67	67.69	66.67	57.59	35.00
GPT-5.5	30.00	64.51	58.89	43.96	11.67
GPT-5.4	25.00	70.54	58.33	45.46	10.00
GLM-5.1	28.33	70.19	68.33	61.09	40.00
Kimi K2.6	31.67	69.25	72.78	66.35	53.33
DeepSeek-V4-Pro	11.67	62.21	57.78	45.21	18.33
DeepSeek-V4-Flash	15.00	66.45	58.89	44.37	11.67
Doubao-Seed-2.0-Pro	6.67	75.93	66.67	55.23	20.00
Doubao-Seed-2.0-Lite	8.33	66.55	62.22	46.97	11.67
Doubao-Seed-2.0-Mini	6.67	72.04	55.00	40.24	5.00
Qwen3.7 Max	31.67	66.12	65.56	58.58	41.67
Qwen3.7 Plus	30.00	68.77	73.33	65.80	50.00
Qwen3.5-27B	18.33	68.39	64.44	56.57	28.33
Qwen3.5-9B	11.67	67.78	63.33	48.79	15.00
Qwen3.5-4B	11.67	54.40	52.78	34.77	8.33

introduced later. Details of the construction process are provided in Appendix B. The resulting set covers both Chinese and English and contains three types of instances: implicit conflict, explicit revision, and normal dialogue. The statistics of WildChat-UC are summarized in Table 2.

We evaluate the selected LLMs under two protocols.

Response-based evaluation. This protocol evaluates implicit conflict handling based on model responses. Given a dialogue history and a follow-up user utterance labeled as an implicit conflict, the evaluated LLM first generates a response without receiving any additional instructions. Subsequently, DeepSeek-V4-Flash is employed to evaluate whether the generated response accurately identifies the implicit conflict. A response is deemed correct only if it recognizes the presence of the conflict; otherwise, it is classified as incorrect. This evaluation protocol is exclusively applied to implicit conflict instances in WildChat-UC, with accuracy adopted as the evaluation metric.

Classification-based evaluation. This protocol formulates the task as three-way classification. Given a dialogue history and the current user utterance, an LLM assigns the utterance to one of three categories: implicit conflict, explicit revision, or normal dialogue. We report Precision, Recall, and F1, together with Recall for the implicit conflict class.

We summarize the main findings as follows.

Finding 1: LLMs rarely recognize implicit conflicts proactively during open-ended response generation. The response-based evaluation shows that, when LLMs are not explicitly instructed to detect implicit conflicts, they tend to directly follow the current user utterance rather than identify its potential incompatibility with prior requirements. As shown in Table 1, the 16 evaluated models achieve an average accuracy of only 19.79% on implicit conflict instances,

with the best model reaching 31.67%. Moreover, 8 of the 16 models obtain an accuracy no higher than 20%.

Finding 2: LLMs show limited recall for implicit conflicts and tend to misclassify them as normal dialogue. Compared with the response-based evaluation, LLMs achieve better overall performance under the classification-based protocol, yet their ability to identify implicit conflicts remains limited. As shown in Table 1, the 16 evaluated models achieve an average IC recall of only 25.00%, with the best model reaching 53.33%; 9 of the 16 models achieve an IC recall no higher than 20%. Although the average F1 reaches 52.04%, this overall score masks the models' weak performance on the implicit conflict class. To further analyze the source of low recall, we present the confusion matrices of four representative models, Claude Opus 4.8, GPT-5.5, Kimi K2.6, and Qwen3.5-4B, in Figure 8. The results show that true implicit conflict instances are frequently misclassified as normal dialogue. This indicates that LLMs often fail to capture the implicit incompatibility between the current utterance and the dialogue history, instead treating it as a natural continuation consistent with the context.

Finding 3: Response-based recognition is strongly associated with classification-based recognition, but the two are not equivalent. We further examine the relationship between response-based accuracy and IC recall under classification-based evaluation. As shown in Figure 9, the two metrics are positively correlated across models, with Pearson's $r = 0.7223$ ($p = 0.0016$) and Spearman's $\rho = 0.6696$ ($p = 0.0045$). The corresponding linear regression yields $R^2 = 0.5217$. These results indicate that models that more reliably recognize implicit conflicts during open-ended response generation also tend to identify them more accurately under classification. However, the two abilities should not be conflated, as their linear relationship explains only 52.17% of the cross-model variation in response-based accuracy. Other factors, such as response strategies and instruction-following behavior, may also affect response-based recognition. Therefore, classification-based evaluation provides a complementary perspective on implicit conflict recognition and supports the development of a dedicated implicit conflict detector.

Finding 4: LLMs struggle with implicit conflict recognition largely because they fail to identify the correct conflict rationale. During annotation, we retain a conflict rationale for each implicit conflict instance and use it as the gold rationale. To better understand the low IC recall, we examine whether the model-generated rationale covers the corresponding gold rationale. Figure 10 shows the joint distribution of label predictions and rationale coverage for true implicit conflict instances. When the model-generated rationale misses the gold rationale, these instances are frequently misclassified as normal dialogue; when the gold rationale is covered, they are more likely to be correctly classified as implicit conflict. Figure 11 further shows that incorrect predictions are concentrated among instances with missing rationales, indicating a strong association between rationale recovery and label correctness. Nevertheless, Qwen3.5-4B